

Modelo de Banco de Dados Relacional (PostgreSQL + PostGIS)

Consolidação de UCAT_PJ, UCMT_PJ, UCBT e das camadas geográficas da BDGD — São José dos Campos e Caçapava

1. Visão geral do modelo

O modelo reúne as nove tabelas já validadas nos quatro documentos anteriores: três de consumidores (ucat_pj, ucmt_pj, ucbt) e seis de infraestrutura física (sub, untrat, untrmt, ssdat, ssdmt, ssdbt). A novidade em relação ao documento consolidado anterior: com sub, untrat e untrmt carregados, deixa de ser necessário usar só CONJ como chave de agrupamento solto — agora existem tabelas reais de subestação e transformador para apontar chaves estrangeiras (FOREIGN KEY) de verdade, então o banco passa a ter integridade referencial de fato, não só campos coincidentes.

1.1 Hierarquia de dependência entre as tabelas

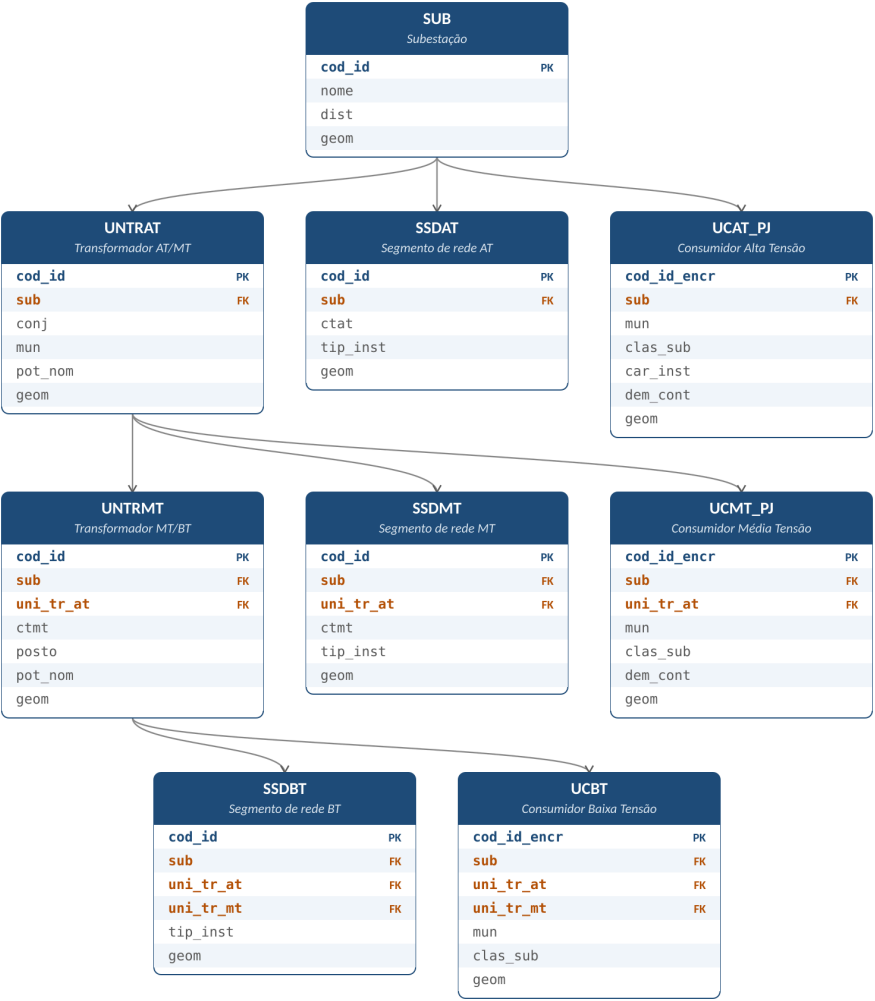
```
sub      (raiz – subestação)
├── untrat  transformador AT/MT
│   └── untrmt  transformador MT/BT
├── ssdat   segmento de AT
├── ssdmt   segmento de MT
├── ssdbt   segmento de BT
├── ucat_pj  consumidor AT
├── ucmt_pj  consumidor MT
└── ucbt    consumidor BT
```

Tabela	Chaves estrangeiras (FK)
untreat	sub → sub.cod_id
untrmt	sub → sub.cod_id uni_tr_at → untrat.cod_id
ssdat	sub → sub.cod_id
ssdmt	sub → sub.cod_id uni_tr_at → untrat.cod_id
ssdbt	sub, uni_tr_at, uni_tr_mt → sub / untrat / untrmt
ucat_pj	sub → sub.cod_id
ucmt_pj	sub → sub.cod_id uni_tr_at → untrat.cod_id
ucbt	sub, uni_tr_at, uni_tr_mt → sub / untrat / untrmt

Ou seja: sub é a raiz de tudo; untrat e untrmt são as tabelas 'ponte' que permitem consultar, com um JOIN de verdade (não uma aproximação por CONJ), todos os consumidores e trechos de rede que dependem eletricamente de um mesmo transformador ou subestação.

1.2 Diagrama entidade-relacionamento (modelo lógico)

Cada caixa traz o nome técnico da tabela, o nome descritivo do que ela representa, e as colunas com PK (chave primária) e FK (chave estrangeira) destacadas. As setas seguem a mesma hierarquia da Seção 1.1.



2. Script de criação do banco (DDL)

Pressupõe PostgreSQL com a extensão PostGIS já habilitada (Seção 4 explica como instalar). Todas as coordenadas ficam em SIRGAS 2000 geográfico (EPSG:4674), o mesmo CRS em que os dados chegaram.

2.1 Extensão e tabela raiz

```
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE sub (
  cod_id      VARCHAR(20) PRIMARY KEY,
  nome       VARCHAR(100),
  dist       VARCHAR(10),
  geom       GEOMETRY(MultiPolygon, 4674)
);
CREATE INDEX idx_sub_geom ON sub USING GIST (geom);
```

2.2 Transformadores (untrat, untrmt)

```
CREATE TABLE untrat (
  cod_id      VARCHAR(20) PRIMARY KEY,
  sub         VARCHAR(20) REFERENCES sub(cod_id),
  conj       VARCHAR(20),
  mun        VARCHAR(10),
  are_loc     VARCHAR(5),
  pot_nom     NUMERIC(10,3),
  geom       GEOMETRY(Point, 4674)
);
CREATE INDEX idx_untrat_geom ON untrat USING GIST (geom);
CREATE INDEX idx_untrat_sub  ON untrat (sub);
CREATE INDEX idx_untrat_conj ON untrat (conj);

CREATE TABLE untrmt (
  cod_id      VARCHAR(20) PRIMARY KEY,
  sub         VARCHAR(20) REFERENCES sub(cod_id),
  uni_tr_at   VARCHAR(20) REFERENCES untrat(cod_id),
  ctmt       VARCHAR(20),
  conj       VARCHAR(20),
  mun        VARCHAR(10),
  are_loc     VARCHAR(5),
  posto      VARCHAR(5),
  pot_nom     NUMERIC(10,3),
  ten_lin_se  NUMERIC(6,3),
  geom       GEOMETRY(Point, 4674)
);
CREATE INDEX idx_untrmt_geom ON untrmt USING GIST (geom);
CREATE INDEX idx_untrmt_sub  ON untrmt (sub);
CREATE INDEX idx_untrmt_uni_tr_at ON untrmt (uni_tr_at);
CREATE INDEX idx_untrmt_conj ON untrmt (conj);
```

2.3 Segmentos de rede (ssdat, ssdmt, ssdbt)

```
CREATE TABLE ssdat (
  cod_id      VARCHAR(20) PRIMARY KEY,
  sub         VARCHAR(20) REFERENCES sub(cod_id),
  conj       VARCHAR(20),
  ctat       VARCHAR(20),
  are_loc     VARCHAR(5),
  tip_inst    VARCHAR(15),
  geom       GEOMETRY(MultiLineString, 4674)
);
CREATE INDEX idx_ssdat_geom ON ssdat USING GIST (geom);
CREATE INDEX idx_ssdat_sub  ON ssdat (sub);

CREATE TABLE ssdmt (
  cod_id      VARCHAR(20) PRIMARY KEY,
  sub         VARCHAR(20) REFERENCES sub(cod_id),
  uni_tr_at   VARCHAR(20) REFERENCES untrat(cod_id),
  ctmt       VARCHAR(20),
```

```

    conj          VARCHAR(20),
    are_loc        VARCHAR(5),
    tip_inst       VARCHAR(15),
    geom           GEOMETRY(MultiLineString, 4674)
);
CREATE INDEX idx_ssdmt_geom ON ssdmt USING GIST (geom);
CREATE INDEX idx_ssdmt_sub  ON ssdmt (sub);
CREATE INDEX idx_ssdmt_uni_tr_at ON ssdmt (uni_tr_at);

CREATE TABLE ssdbt (
    cod_id         VARCHAR(20) PRIMARY KEY,
    sub            VARCHAR(20) REFERENCES sub(cod_id),
    uni_tr_at      VARCHAR(20) REFERENCES untrat(cod_id),
    uni_tr_mt      VARCHAR(20) REFERENCES untrmt(cod_id),
    ctmt          VARCHAR(20),
    conj          VARCHAR(20),
    are_loc        VARCHAR(5),
    tip_inst       VARCHAR(15),
    geom           GEOMETRY(MultiLineString, 4674)
);
CREATE INDEX idx_ssdbt_geom ON ssdbt USING GIST (geom);
CREATE INDEX idx_ssdbt_sub  ON ssdbt (sub);
CREATE INDEX idx_ssdbt_uni_tr_mt ON ssdbt (uni_tr_mt);

```

2.4 Consumidores (ucat_pj, ucmt_pj, ucbt)

Já filtrados por SIT_ATIV = 'AT' e pelas duas cidades antes da carga — por isso sit_ativ não entra como coluna aqui (Seção 3 dos documentos anteriores).

```
CREATE TABLE ucat_pj (
  cod_id_encr    VARCHAR(80) PRIMARY KEY,
  mun            VARCHAR(10),
  brr            VARCHAR(60),
  sub            VARCHAR(20) REFERENCES sub(cod_id),
  conj           VARCHAR(20),
  ctat           VARCHAR(20),
  clas_sub       VARCHAR(10),
  cnae           VARCHAR(10),
  car_inst       NUMERIC(15,3),
  dem_cont       NUMERIC(15,3),
  tip_sist       VARCHAR(15),
  are_loc        VARCHAR(5),
  geom           GEOMETRY(Point, 4674)
);
CREATE INDEX idx_ucat_geom ON ucat_pj USING GIST (geom);
CREATE INDEX idx_ucat_sub  ON ucat_pj (sub);
CREATE INDEX idx_ucat_conj ON ucat_pj (conj);
CREATE INDEX idx_ucat_mun  ON ucat_pj (mun);

CREATE TABLE ucmt_pj (
  cod_id_encr    VARCHAR(80) PRIMARY KEY,
  mun            VARCHAR(10),
  brr            VARCHAR(60),
  sub            VARCHAR(20) REFERENCES sub(cod_id),
  uni_tr_at      VARCHAR(20) REFERENCES untrat(cod_id),
  conj           VARCHAR(20),
  ctmt           VARCHAR(20),
  clas_sub       VARCHAR(10),
  cnae           VARCHAR(10),
  car_inst       NUMERIC(15,3),
  dem_cont       NUMERIC(15,3),
  tip_sist       VARCHAR(15),
  are_loc        VARCHAR(5),
  geom           GEOMETRY(Point, 4674)
);
CREATE INDEX idx_ucmt_geom ON ucmt_pj USING GIST (geom);
CREATE INDEX idx_ucmt_sub  ON ucmt_pj (sub);
CREATE INDEX idx_ucmt_uni_tr_at ON ucmt_pj (uni_tr_at);
CREATE INDEX idx_ucmt_conj ON ucmt_pj (conj);
CREATE INDEX idx_ucmt_mun  ON ucmt_pj (mun);

CREATE TABLE ucbt (
  cod_id_encr    VARCHAR(80) PRIMARY KEY,
  mun            VARCHAR(10),
  brr            VARCHAR(60),
  sub            VARCHAR(20) REFERENCES sub(cod_id),
  uni_tr_at      VARCHAR(20) REFERENCES untrat(cod_id),
  uni_tr_mt      VARCHAR(20) REFERENCES untrmt(cod_id),
  conj           VARCHAR(20),
  ctmt           VARCHAR(20),
  clas_sub       VARCHAR(10),
  cnae           VARCHAR(10),
  car_inst       NUMERIC(15,3),
  tip_sist       VARCHAR(15),
  are_loc        VARCHAR(5),
  geom           GEOMETRY(Point, 4674)
);
CREATE INDEX idx_ucbt_geom ON ucbt USING GIST (geom);
CREATE INDEX idx_ucbt_sub  ON ucbt (sub);
CREATE INDEX idx_ucbt_uni_tr_mt ON ucbt (uni_tr_mt);
CREATE INDEX idx_ucbt_conj ON ucbt (conj);
CREATE INDEX idx_ucbt_mun  ON ucbt (mun);
```

Por que VARCHAR nos códigos (cod_id, sub, conj, mun, ctat, ctmt) mesmo quando parecem números: são identificadores, não quantidades — evita perder zero à esquerda e impede que alguém tente somar ou calcular médias com um código por engano.

2.5 DLL Completa

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

```
-- Subestação
```

```
CREATE TABLE sub (  
  cod_id   VARCHAR(20) PRIMARY KEY,  
  nome     VARCHAR(100),  
  dist     VARCHAR(10),  
  geom     GEOMETRY(MultiPolygon, 4674)  
);  
CREATE INDEX idx_sub_geom ON sub USING GIST (geom);
```

```
-- Transformadores
```

```
CREATE TABLE untrat (  
  cod_id   VARCHAR(20) PRIMARY KEY,  
  sub      VARCHAR(20) REFERENCES sub(cod_id),  
  conj     VARCHAR(20),  
  mun      VARCHAR(10),  
  are_loc  VARCHAR(5),  
  pot_nom  NUMERIC(10,3),  
  geom     GEOMETRY(Point, 4674)  
);  
CREATE INDEX idx_untrat_geom ON untrat USING GIST (geom);  
CREATE INDEX idx_untrat_sub  ON untrat (sub);  
CREATE INDEX idx_untrat_conj ON untrat (conj);
```

```
CREATE TABLE untrmt (  
  cod_id   VARCHAR(20) PRIMARY KEY,  
  sub      VARCHAR(20) REFERENCES sub(cod_id),  
  uni_tr_at VARCHAR(20) REFERENCES untrat(cod_id),  
  ctmt     VARCHAR(20),  
  conj     VARCHAR(20),  
  mun      VARCHAR(10),  
  are_loc  VARCHAR(5),  
  posto    VARCHAR(5),  
  pot_nom  NUMERIC(10,3),  
  ten_lin_se NUMERIC(6,3),  
  geom     GEOMETRY(Point, 4674)  
);  
CREATE INDEX idx_untrmt_geom ON untrmt USING GIST (geom);  
CREATE INDEX idx_untrmt_sub  ON untrmt (sub);  
CREATE INDEX idx_untrmt_uni_tr_at ON untrmt (uni_tr_at);  
CREATE INDEX idx_untrmt_conj ON untrmt (conj);
```

```
-- Segmentos de rede
```

```
CREATE TABLE ssdat (  
  cod_id   VARCHAR(20) PRIMARY KEY,
```

```

    sub    VARCHAR(20) REFERENCES sub(cod_id),
    conj   VARCHAR(20),
    ctat   VARCHAR(20),
    are_loc VARCHAR(5),
    tip_inst VARCHAR(15),
    geom    GEOMETRY(MultiLineString, 4674)
);
CREATE INDEX idx_ssdat_geom ON ssdat USING GIST (geom);
CREATE INDEX idx_ssdat_sub ON ssdat (sub);

CREATE TABLE ssdmt (
    cod_id   VARCHAR(20) PRIMARY KEY,
    sub      VARCHAR(20) REFERENCES sub(cod_id),
    uni_tr_at VARCHAR(20) REFERENCES untrat(cod_id),
    ctmt     VARCHAR(20),
    conj     VARCHAR(20),
    are_loc  VARCHAR(5),
    tip_inst VARCHAR(15),
    geom     GEOMETRY(MultiLineString, 4674)
);
CREATE INDEX idx_ssdmt_geom ON ssdmt USING GIST (geom);
CREATE INDEX idx_ssdmt_sub ON ssdmt (sub);
CREATE INDEX idx_ssdmt_uni_tr_at ON ssdmt (uni_tr_at);

CREATE TABLE ssdbt (
    cod_id   VARCHAR(20) PRIMARY KEY,
    sub      VARCHAR(20) REFERENCES sub(cod_id),
    uni_tr_at VARCHAR(20) REFERENCES untrat(cod_id),
    uni_tr_mt VARCHAR(20) REFERENCES untrmt(cod_id),
    ctmt     VARCHAR(20),
    conj     VARCHAR(20),
    are_loc  VARCHAR(5),
    tip_inst VARCHAR(15),
    geom     GEOMETRY(MultiLineString, 4674)
);
CREATE INDEX idx_ssdbt_geom ON ssdbt USING GIST (geom);
CREATE INDEX idx_ssdbt_sub ON ssdbt (sub);
CREATE INDEX idx_ssdbt_uni_tr_mt ON ssdbt (uni_tr_mt);

-- Consumidores

CREATE TABLE ucat_pj (
    cod_id_encr VARCHAR(80) PRIMARY KEY,
    mun         VARCHAR(10),
    brr         VARCHAR(60),
    sub         VARCHAR(20) REFERENCES sub(cod_id),
    conj        VARCHAR(20),
    ctat        VARCHAR(20),
    clas_sub    VARCHAR(10),
    cnae        VARCHAR(10),
    car_inst    NUMERIC(15,3),
    dem_cont    NUMERIC(15,3),
    tip_sist    VARCHAR(15),
    are_loc     VARCHAR(5),
    geom        GEOMETRY(Point, 4674)
);
CREATE INDEX idx_ucat_geom ON ucat_pj USING GIST (geom);

```

```
CREATE INDEX idx_ucat_sub ON ucat_pj (sub);
CREATE INDEX idx_ucat_conj ON ucat_pj (conj);
CREATE INDEX idx_ucat_mun ON ucat_pj (mun);
```

```
CREATE TABLE ucmt_pj (
  cod_id_encr VARCHAR(80) PRIMARY KEY,
  mun VARCHAR(10),
  brr VARCHAR(60),
  sub VARCHAR(20) REFERENCES sub(cod_id),
  uni_tr_at VARCHAR(20) REFERENCES untrat(cod_id),
  conj VARCHAR(20),
  ctmt VARCHAR(20),
  clas_sub VARCHAR(10),
  cnae VARCHAR(10),
  car_inst NUMERIC(15,3),
  dem_cont NUMERIC(15,3),
  tip_sist VARCHAR(15),
  are_loc VARCHAR(5),
  geom GEOMETRY(Point, 4674)
);
CREATE INDEX idx_ucmt_geom ON ucmt_pj USING GIST (geom);
CREATE INDEX idx_ucmt_sub ON ucmt_pj (sub);
CREATE INDEX idx_ucmt_uni_tr_at ON ucmt_pj (uni_tr_at);
CREATE INDEX idx_ucmt_conj ON ucmt_pj (conj);
CREATE INDEX idx_ucmt_mun ON ucmt_pj (mun);
```

```
CREATE TABLE ucbt (
  cod_id_encr VARCHAR(80) PRIMARY KEY,
  mun VARCHAR(10),
  brr VARCHAR(60),
  sub VARCHAR(20) REFERENCES sub(cod_id),
  uni_tr_at VARCHAR(20) REFERENCES untrat(cod_id),
  uni_tr_mt VARCHAR(20) REFERENCES untrmt(cod_id),
  conj VARCHAR(20),
  ctmt VARCHAR(20),
  clas_sub VARCHAR(10),
  cnae VARCHAR(10),
  car_inst NUMERIC(15,3),
  tip_sist VARCHAR(15),
  are_loc VARCHAR(5),
  geom GEOMETRY(Point, 4674)
);
CREATE INDEX idx_ucbt_geom ON ucbt USING GIST (geom);
CREATE INDEX idx_ucbt_sub ON ucbt (sub);
CREATE INDEX idx_ucbt_uni_tr_mt ON ucbt (uni_tr_mt);
CREATE INDEX idx_ucbt_conj ON ucbt (conj);
CREATE INDEX idx_ucbt_mun ON ucbt (mun);
```

3. Como instalar e habilitar o PostGIS

3.1 Instalação

- Linux (Ubuntu/Debian): `sudo apt install postgresql postgresql-contrib postgis`
- macOS (Homebrew): `brew install postgresql postgis`
- Windows: instalador oficial do PostgreSQL (postgresql.org/download) já inclui o Stack Builder, onde dá para marcar o PostGIS como componente adicional.
- Alternativa mais simples para começar (qualquer sistema operacional): usar a imagem Docker oficial `postgis/postgis`, que já vem com tudo pronto — dispensa instalação manual.

```
docker run --name bdgd-db -e POSTGRES_PASSWORD=suasenha -p 5432:5432 -d postgis/postgis:16-3.4
```

3.2 Criar o banco e habilitar a extensão

```
-- Database: bdgd
```

```
-- DROP DATABASE IF EXISTS bdgd;
```

```
CREATE DATABASE bdgd
```

```
WITH
```

```
OWNER = postgres
```

```
ENCODING = 'UTF8'
```

```
LC_COLLATE = 'Portuguese_Brazil.1252'
```

```
LC_CTYPE = 'Portuguese_Brazil.1252'
```

```
LOCALE_PROVIDER = 'libc'
```

```
TABLESPACE = pg_default
```

```
CONNECTION LIMIT = -1
```

```
IS_TEMPLATE = False;
```

4. Como carregar os dados

5. Provando que o banco 'conversa' — consultas de exemplo

5.1 Todos os consumidores dependentes de uma subestação, nos três níveis de tensão

```
SELECT 'AT' AS nivel, cod_id_encr, geom FROM ucat_pj WHERE sub = 'SJC'
UNION ALL
SELECT 'MT' AS nivel, cod_id_encr, geom FROM ucmt_pj WHERE sub = 'SJC'
UNION ALL
SELECT 'BT' AS nivel, cod_id_encr, geom FROM ucbt WHERE sub = 'SJC';
```

5.2 Transformadores MT/BT e o transformador AT/MT de origem, com distância real em metros

```
SELECT b.cod_id AS transformador_mt_bt, a.cod_id AS transformador_at_mt,
       ST_Distance(ST_Transform(b.geom, 31983), ST_Transform(a.geom, 31983)) AS distancia_m
FROM untrmt b
JOIN untrat a ON b.uni_tr_at = a.cod_id
ORDER BY distancia_m DESC
LIMIT 10;
```

5.3 Consumidores de BT dentro de um raio de cobertura de RF (ex.: 300 m) de cada transformador

```
SELECT t.cod_id AS transformador, COUNT(c.cod_id_encr) AS consumidores_no_raio
FROM untrmt t
JOIN ucbt c
    ON ST_DWithin(ST_Transform(t.geom, 31983), ST_Transform(c.geom, 31983), 300)
GROUP BY t.cod_id
ORDER BY consumidores_no_raio DESC;
```

As duas primeiras usam o relacionamento por chave estrangeira (JOIN normal); a terceira usa o PostGIS de verdade (ST_DWithin, ST_Transform para metros) — exatamente o tipo de consulta que sustenta o motor de cálculo de cobertura do projeto.

6. Resumo executivo

- Nove tabelas: sub (raiz), untrat e untrmt (transformadores, tabelas-ponte), ssdat/ssdmt/ssdbt (segmentos de rede) e ucat_pj/ucmt_pj/ucbt (consumidores) — todas as colunas essenciais já definidas nos quatro documentos anteriores.
- Diferente do que foi dito no documento consolidado anterior (quando só tínhamos os CSVs), agora com sub/untrat/untrmt carregados existem FOREIGN KEYS reais entre os três níveis de tensão, não só uma chave de agrupamento solta por CONJ.
- PostGIS escolhido corretamente para o caso: todas as tabelas têm coluna geom nativa, com índice espacial GIST, permitindo consultas de distância e raio de cobertura direto no banco (ST_DWithin, ST_Distance), sem precisar exportar para outra ferramenta.
- Carga recomendada via ogr2ogr para as camadas geográficas (GeoPackage) e via staging + ST_MakePoint para os CSVs de consumidores.